ANNEXE : Les méthodes et les propriétés à utiliser avec la bibliothèque pandas

Catégorie	Syntaxe
Importation des bibliothèques	▪ **import** nom_bibliothèque **as** alias ▪ **from** nom_bibliothèque **import** élément(s)
Création d'une Series	▪ Id_Serie **=** alias**.Series ([** val1, …, valN **])** ▪ Id_Serie **=** alias**.Series ([** Val1, …, ValN **] , index = [** Idx1, …, IdxN **])**
Affichage des données d'une Series	▪ **print (** Id_Serie **)** ▪ **print (** Id_Serie **[** Indice **])** *ou bien* **print (** Id_Serie **[[** Indice **]])** ▪ **print (** Id_Serie **[[** Indice1**,** Indice2**,** …**]])** ▪ **print (** Id_Serie **[** Indice1 **:** Indice2 **])**
Modification des valeurs d'une Serie	▪ Id_Serie **[** Indice **] =** Valeur ▪ Id_Serie **[[** Indice1, …, IndiceN **]] = [** Valeur1, …, ValeurN **]**
Filtrage des données d'une Series	▪ **print (** Id_Serie **[** Condition **])** ▪ **print (** Id_Serie **[** Condition1 **Operateur_Logique** Condition2 **])**
Création d'un DataFrame	▪ Id_DataFrame **=** alias**.DataFrame (columns = [**Colonne1, …, ColonneN**] , index = [**Idx1, …, IdxM**] , data = [[**Val11, …, Val1N**]**, …, **[**ValM1, …,ValMN**]])** ▪ Id_DataFrame **=** alias**.DataFrame ({** Id_Colonne1 **: [**Val1, …, ValN**] ,** … **,** Id_ColonneM **: [**Val1, …, ValN**] })** ▪ Id_DataFrame **=** alias**.DataFrame ({** Id_Colonne1 **: [**Val1, …, ValN**] ,** … **,** Id_ColonneM **: [**Val1, …, ValN**] } , index = [**Idx1, Idx2, …, IdxN**])**
Importation des données dans un DataFrame	▪ Id_DataFrame **=** alias**.read_excel (** 'chemin/nom_fichier.xlsx' **,** 'Nom_Feuille' **, index_col =** N°_Colonne **, header =** 0 **)** ▪ Id_DataFrame **=** alias**.read_csv (** 'chemin/nom_fichier.csv' **, sep =** 'séparateur' **, index_col =** N°_colonne **, header =** 0 **)** ▪ Id_DataFrame **=** alias**.read_json (** 'chemin/nom_fichier.json' **)**
Manipulation d'un DataFrame	▪ **print (** Id_DataFrame**.shape)** ▪ **print (** Id_DataFrame**.size)** ▪ **print (** Id_DataFrame.**info ( ))** ▪ **print (** Id_DataFrame.**describe ( ))** ▪ **print (** Id_DataFrame**.columns)** ▪ Resultat **=** Id_DataFrame**.rename (columns = {** ancien_nom **:** nouveau_nom **,** … **} , index = {** ancien_indice **:** nouveau_indice **,** …**})** ▪ Resultat **=** Id_DataFrame**.drop (columns = [** "Nom_Colonne1",…,"Nom_ColonneN" **])** *ou bien* Resultat **=** Id_DataFrame**.drop ([** "Nom_Colonne1",…,"Nom_ColonneN" **] , axis = 1)** ▪ Id_Series **=** Id_DataFrame**.pop (** "Nom_Colonne" **)** ▪ Resultat **=** Id_DataFrame**.drop ([** Indice_Ligne1, … , Indice_LigneN **])** ▪ Resultat **=** Id_DataFrame**.drop (** Id_DataFrame**.index [** Indice_Ligne_Initial **:** Indice_Ligne_Final **])**

Catégorie	Syntaxe
Affichage des données d'un DataFrame	▪ **print (** Id_DataFrame **)** ▪ **print (** Id_DataFrame.Nom_Colonne **)** *ou bien* **print (** Id_DataFrame **[** "Nom_Colonne" **])** ▪ **print (** Id_DataFrame **[[**"Nom_Colonne1"**,** "Nom_Colonne2"**, …]])** ▪ **print (** Id_DataFrame **[**"Nom_Colonne"**] [**N° LigneInitial **:** N° LigneFinal **])** ▪ **print (** Id_DataFrame**.loc [** Indice_Ligne **])** ▪ **print (** Id_DataFrame**.head (** n **))** ▪ **print (** Id_DataFrame**.tail (** n **))** ▪ **print (** Id_DataFrame**.loc [** Indice_LigneDeb **:** Indice_LigneFin **])** ▪ **print (** Id_DataFrame**.loc [** Indice_Ligne **,** Nom_Colonne **])**
Modification et ajout des données dans un DataFrame	▪ Id_DataFrame**.loc [Indice_Ligne , Nom_Colonne] = Valeur (ou Formule)** ▪ Id_DataFrame**.loc [Indice_Ligne] = [Liste_Valeur]** ▪ IdDataFrame **["Nom_Colonne"] = Valeur (ou Formule)**
Nettoyage d'un DataFrame	▪ Resultat **=** Id_DataFrame**.drop_duplicates ( )** ▪ Resultat **=** Id_DataFrame**.dropna ( )** ▪ Resultat **=** Id_DataFrame**.dropna (axis = 1)**
Les fonctions statistiques	Liste des fonctions : **mean ( )** – **min ( )** – **max ( )** – **sum ( )** – **count ( )** Syntaxe sur un DataFrame : **print (** Id_DataFrame**.Nom_Fonction ( ))** Syntaxe sur une colonne d'un DataFrame : **print (** Id_DataFrame **["Nom_Colonne"].Nom_Fonction ( ))**
Tri des éléments d'un DataFrame	▪ Resultat **=** Id_DataFrame**.sort_index ( )** ▪ Resultat **=** Id_DataFrame**.sort_index (axis=1)** ▪ Resultat **=** Id_DataFrame**.sort_values (by = ["Nom_Colonne1" ,"Nom_Colonne2" , …] , ascending = [True/False , True/False , …])**
Affichage des données d'un DataFrame selon une ou plusieurs conditions	▪ **print (** Id_DataFrame **[** Id_DataFrame **["Nom_Colonne"]** OperateurComparaison **Valeur])** ▪ **print (** Id_DataFrame **[["Nom_Colonne1", "Nom_Colonne2", …]] [** Id_DataFrame **["Nom_Colonne"]** Op_Comp **Valeur])** ▪ **print (** Id_DataFrame **[(**Id_DataFrame**["Nom_Colonne"]** Op_Comp **Valeur)** Op_Logique **(Id_DataFrame ["Nom_Colonne"]** Op_Comp **Valeur)]** LES OPERATEURS DE COMPARAISON **==** | **<** | **<=** | **>** | **>=** | **!=** LES OPERATEURS LOGIQUES Opérateur | Nomination | Rôle | | OU logique | Vérifie qu'une des conditions est réalisée & | ET logique | Vérifie que toutes les conditions sont réalisées

Catégorie	Syntaxe
Ajout d'une colonne remplie par des données suite à une condition simple	**import numpy as** alias Id_DataFrame **["Nouvelle_Colonne"] =** alias**.where (** Id_DataFrame **["Nom_Colonne"]** Op_Comp **Valeur ,** Valeur_Si_Vrai **,** Valeur_Si_Faux **)**
Création d'un graphique	**import matplotlib.pyplot as** alias Id_DataFrame**.plot.Type_Graphique (x =** "Nom_Colonne_Abscisse" **, y =** "Nom_Colonne_Ordonnée" **, title =** "Titre_Graphique" **,** **color =** "Couleur_Graphique" **, xlabel =** "Titre_Axe_Abcisse" **, ylabel =** "Titre_Axe_Ordonnée" **, legend =** True/False **,** **figsize = (**Longueur**,** Largeur**) , rot =** Degré_Rotation_Étiquettes **)** alias**.tight_layout ( )** alias**.savefig (** "Nom_Figure.png" **)** alias**.show ( )** ***Type_Graphique*** **: line()** → courbe, **bar()** → barres, **scatter()** → dispersion, **hist()** → distribution, **pie()** → proportion
Regroupement / Agrégation	***Syntaxe avec une seule colonne de groupement :*** Resultat **=** Id_DataFrame**.groupby (** "Colonne_de_Groupement" **) [** "Colonne_à_Agréger" **] .** Fonction_Statistique **( )** ***Syntaxe avec plusieurs colonnes de groupement :*** Resultat **=** Id_DataFrame**.groupby ([**"Colonne1_de_Groupement"**, … ,** "ColonneN_de_Groupement" **) [** "Colonne_à_Agréger" **] .** Fonction_Statistique **( )** ***Syntaxe avec plusieurs fonctions d'agrégation :*** Resultat **=** Id_DataFrame**.groupby (** "Colonne_de_Groupement" **) [** "Colonne_à_Agréger" **] . agg ([** Fonction1 **( ) ,** … **,** FonctionN **( )])**